FeedbackLens — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and where to start

The same shape as MeetingMind, three jobs:

- `src/schemas.py` — `Praises(items: List[str] = [])`, `Complaints(items: List[str] = [])` (each wraps a list so `with_structured_output` has a single object to return), and `FeedbackReport(summary, praises, complaints)`.
- `src/report.py` — `compile_report(parts)` (unwrap `parts["praises"].items` / `parts["complaints"].items`, keep `parts["summary"]`) and `format_report(report)` (plain `for` loops). **Build and test this first, with hand-made parts — no model.**
- `src/agents/` — `build_praises_chain`, `build_complaints_chain` (both `with_structured_output`), `build_summary_chain` (`StrOutputParser`). Each takes `{"reviews": ...}`.
- `src/pipeline.py` — `RunnableParallel(summary=..., praises=..., complaints=...)` + `run_analysis`.

Hint 2 — parallel and compile, the key lines

The three chains are independent, so run them together and compile the dict:

```
build_pipeline(model):
    return RunnableParallel(
        summary    = build_summary_chain(model),
        praises    = build_praises_chain(model),
        complaints = build_complaints_chain(model),
    )

run_analysis(model, reviews):
    parts = build_pipeline(model).invoke({"reviews": reviews})
    return compile_report(parts)
```

The keys you give `RunnableParallel` (`summary` / `praises` / `complaints`) must match what `compile_report` reads. Two details: the two list extractors use `with_structured_output` (you loop over the result), the summary uses `StrOutputParser` (it's prose); and every chain takes the **same** `{"reviews": ...}` input, which is what lets them run side by side. The whole file set is in the complete solution.